NCA - Lab report
Discrete Event Simulator for Queueing Systems

Group 10

Abdul Hadi Farooq

April 28, 2026

1 Introduction

The objective of this laboratory is to understand performance evaluation of computer systems through discrete-event simulation of queueing models. This work implements a complete object-oriented discrete-event simulator that models the behavior of message-based systems with queuing and service components, specifically implementing the M/M/1, M/M/1/4, M/M/1/8, and M/M/3/8 queueing models.

Queueing theory is fundamental to performance evaluation of computer systems, communication networks, and data centers. Real systems often operate under randomness: messages arrive at random times (modeled by exponential inter-arrival distributions), and services complete at random durations (modeled by exponential service times). Understanding how these systems perform under varying load conditions is essential for capacity planning and optimization.

This laboratory explores four classic queueing scenarios:

- **M/M/1:** Single server, infinite capacity queue
- **M/M/1/4:** Single server, maximum 4 entities in system (capacity-limited)
- **M/M/1/8:** Single server, maximum 8 entities in system (capacity-limited)
- **M/M/3/8:** Three parallel servers, maximum 8 entities in system

For each model, we evaluate performance across varying arrival rates $\lambda \in \{4, 6, 8, 12\}$ messages per second, with fixed service rate $\mu = 8$ messages per second. Performance metrics include average system occupancy, queue length, waiting time, throughput, server utilization, and loss probability for capacity-limited systems.

1.1 Methodology

- **Programming Language:** Python 3.14 with NumPy (random generation) and Matplotlib (plotting)

- **Simulator Architecture:** Object-oriented design with Message, Event, Scheduler, Client, Queue, Server, and Gateway classes
- **Simulation Parameters:**
  - Simulation time per run: 20,000 seconds
  - Warmup period: 2,000 seconds (10% of simulation time)
  - Independent replications: 30 runs per configuration
  - Total configurations: 16 (4 scenarios $\times$ 4 arrival rates)
  - Total replication runs: 480
- **Statistical Analysis:** 95% confidence intervals computed via t-distribution with Welford's algorithm for numerical stability
- **Reproducibility:** Deterministic seeding based on scenario and replication index ensures reproducible results
- **Event Model:**
  - SEND_MSG: Client sends message (source=client, destination=gateway)
  - RECV_MSG: Gateway receives message (after 1-second propagation delay)
  - MSG_DEPT: Service completion at gateway and message departure from system

1.2 Code Repository

The complete implementation is available at:

`https://github.com/abdulhadihub/cnam-performance-evaluation-course/tree/main/lab-2`

The repository includes:

- `sim/message.py` - Message class with ID, source, destination
- `sim/event.py` - Event class with type (SEND/RECV/DEPT), timestamp, message reference
- `sim/scheduler.py` - Min-heap based scheduler with O(log n) insertion
- `sim/client.py` - Client with exponential inter-arrival generation
- `sim/queue_model.py` - FIFO queue with configurable capacity
- `sim/server.py` - Server with exponential service-time generation
- `sim/gateway.py` - Gateway orchestrating queue+servers, admission control, departure logic
- `sim/engine.py` - Main simulator loop, metrics collection, trace generation, class tests

- `sim/experiments.py` - Batch experiment runner, replication aggregation, CI computation, plotting
- `main.py` - CLI interface for tests, single runs, and experiment sets
- `outputs/` - Generated traces, results, and plots

2 System Architecture

2.1 Design Overview

The simulator follows a classical discrete-event simulation architecture:

1. **Scheduler:** Maintains events in chronological order. Uses a min-heap to achieve O(log n) insertion without full re-sorting.
2. **Event Loop:** Repeatedly extracts the earliest event, updates system state, and may schedule new events.
3. **Components:** Each system component (Client, Queue, Server, Gateway) is modeled as an object with state and methods.
4. **Message Tracking:** Messages carry metadata (ID, source, destination, creation time) to compute response times and queue waits.
5. **Metrics Collection:** After each event, system state is recorded for time-weighted and count-based metric computation.
6. **Trace Generation:** A CSV trace file records every event (time, node, event type, source, destination, message ID).

2.2 Queueing Model Details

2.2.1 Capacity Convention

For M/M/1/K models, capacity K represents the total number of entities allowed in the system (both in queue and in service). This is the standard M/M/1/K convention:

- M/M/1: No capacity limit
- M/M/1/4: Maximum 4 entities total (if all 4 are occupied, new arrivals are dropped)
- M/M/1/8: Maximum 8 entities total
- M/M/3/8: Maximum 8 entities with 3 parallel servers

2.2.2 Arrival Process

Inter-arrival times follow an exponential distribution with mean $1/\lambda$:

$$\text{Inter-arrival time} \sim \text{Exponential}(\lambda) \tag{1}$$

For example, with $\lambda = 6$ messages/s, the mean inter-arrival time is $1/6 \approx 0.167$ seconds.

2.2.3 Service Process

Service times follow an exponential distribution with mean $1/\mu = 1/8 = 0.125$ seconds:

$$\text{Service time} \sim \text{Exponential}(\mu) \tag{2}$$

All servers in all scenarios have the same $\mu = 8$ messages/s, so M/M/3 systems have aggregate service capacity of $3 \times 8 = 24$ messages/s when all three servers are busy.

2.2.4 Queue Discipline

All queues follow FIFO (First-In-First-Out) discipline. When a server becomes idle:

- If queue is not empty: next message from queue begins service immediately
- If queue is empty: server becomes idle, waiting for next arrival

For capacity-limited systems, if the system is full when an arrival occurs, that message is dropped and not served.

3 Metrics

3.1 Time-Weighted Metrics

After warmup period, the simulator computes:

1. **Average System Occupancy:** Mean number of entities in system (queue + in service)

$$N_s = \frac{1}{T} \int_0^T N(t)\, dt \tag{3}$$

   where $N(t)$ is the instantaneous system occupancy and T is the measured time interval after warmup.

2. **Average Queue Length:** Mean number of entities waiting in queue (not in service)

$$N_q = \frac{1}{T} \int_0^T N_q(t)\, dt \tag{4}$$

3. **Average Server Utilization:** Fraction of time at least one server is busy

$$\rho = \frac{\text{Total server busy time}}{T \times \text{Number of servers}} \tag{5}$$

3.2 Count-Based Metrics

1. **Throughput:** Number of completed messages per unit time (after warmup)

$$\text{Throughput} = \frac{\text{Departures after warmup}}{T} \tag{6}$$

2. **Average Queue Waiting Time:** Mean time spent waiting in queue before service

$$W_q = \frac{\sum \text{Wait times}}{N_{\text{departures}}} \tag{7}$$

3. **Average Response Time:** Mean time from arrival to departure

$$W = \frac{\sum(\text{Departure time} - \text{Arrival time})}{N_{\text{departures}}} \tag{8}$$

4. **Drop Probability (capacity-limited systems):** Fraction of arrivals that are dropped

$$P_{\text{drop}} = \frac{\text{Dropped arrivals}}{\text{Total arrivals}} \tag{9}$$

3.3 Confidence Intervals

For each metric and configuration, we compute a two-sided 95% confidence interval using the t-distribution:

$$\text{CI} = \bar{x} \pm t_{0.975}(n-1) \cdot \frac{s}{\sqrt{n}} \tag{10}$$

where:

- $\bar{x}$ is the sample mean across 30 replications
- s is the sample standard deviation
- $t_{0.975}(n-1)$ is the two-sided t critical value (e.g., $t_{0.975}(29) \approx 2.045$)
- $n = 30$ is the number of replications

4 Results

4.1 M/M/1: Unlimited Capacity, Single Server

4.1.1 Theoretical Background

The M/M/1 queue is the simplest and most studied queueing model. Under steady state with $\rho = \lambda/\mu < 1$, theoretical results are well-known:

- Average system occupancy: $N_s = \frac{\rho}{1-\rho}$

- Average queue length: $N_q = \frac{\rho^2}{1-\rho}$
- Average wait in queue: $W_q = \frac{\rho}{\mu(1-\rho)}$
- Average response time: $W = \frac{1}{\mu(1-\rho)}$
- Utilization: $\rho = \lambda/\mu$
- Stability condition: $\rho < 1$ (i.e., $\lambda < \mu$)

For our scenario with $\mu = 8$:

- $\lambda = 4$: $\rho = 0.5$, stable
- $\lambda = 6$: $\rho = 0.75$, stable
- $\lambda = 8$: $\rho = 1.0$, marginally unstable (infinite queue at equilibrium)
- $\lambda = 12$: $\rho = 1.5$, unstable (queue grows without bound)

4.1.2 Average System Occupancy

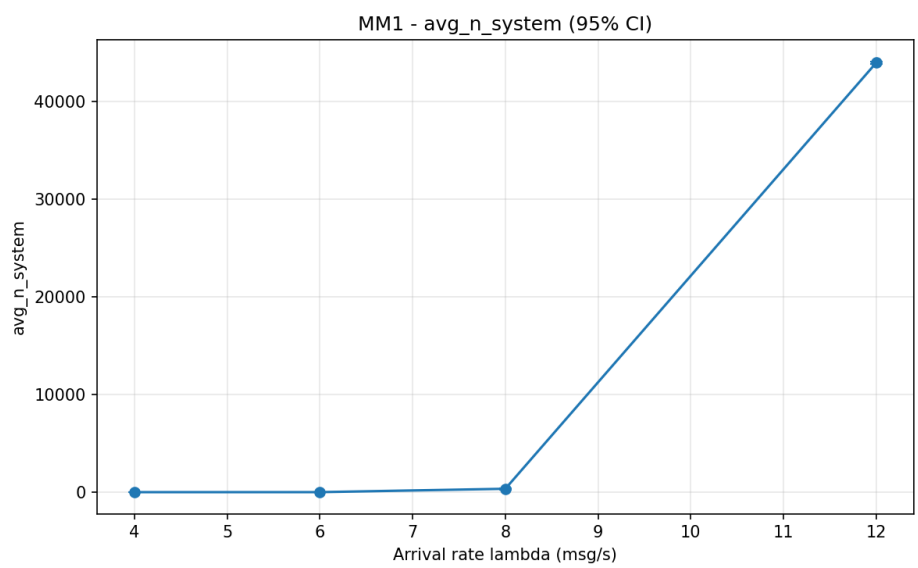

Figure 1: M/M/1: Average system occupancy vs. arrival rate. With 95% CI error bars.

4.1.3 Average Queue Waiting Time

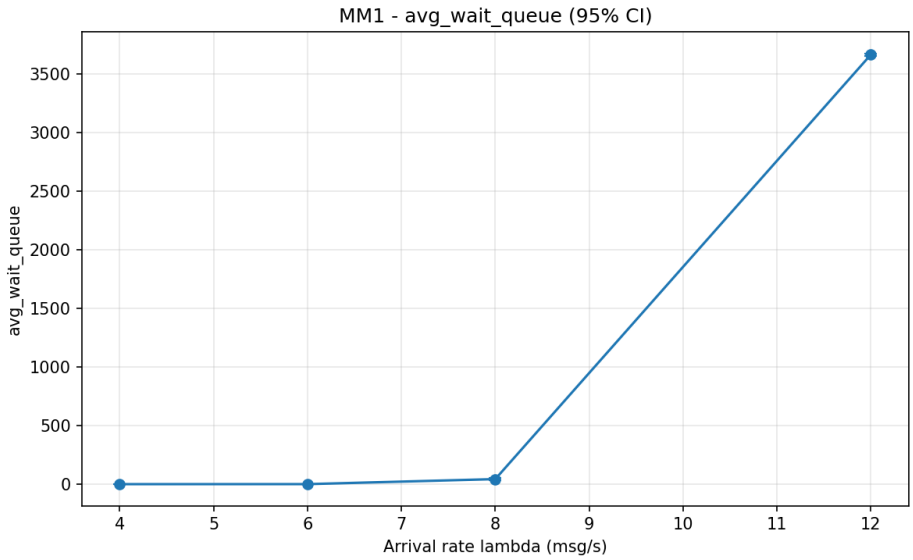

Figure 2: M/M/1: Average queue wait time vs. arrival rate. With 95% CI error bars.

4.1.4 Throughput

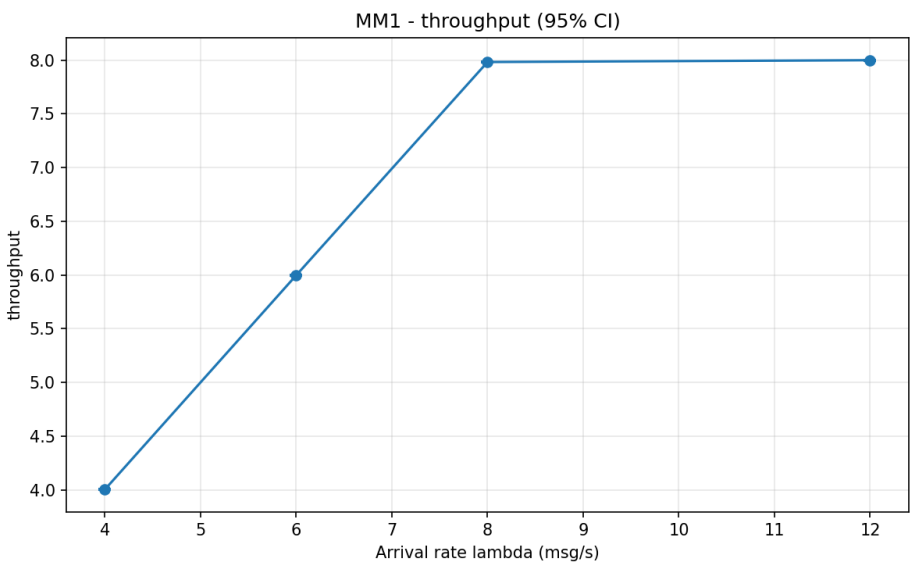

Figure 3: M/M/1: Throughput vs. arrival rate. With 95% CI error bars.

4.1.5 Drop Probability

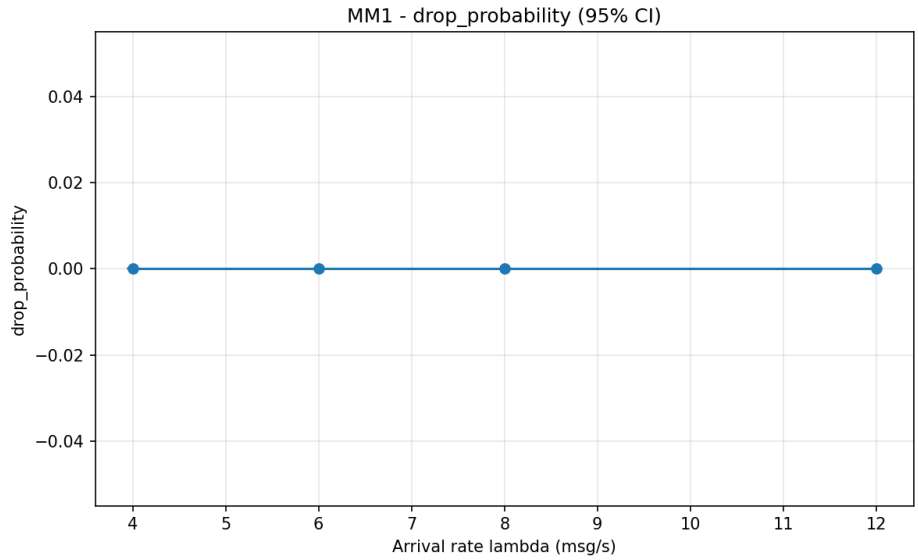

Figure 4: M/M/1: Drop probability vs. arrival rate. With 95% CI error bars. (Always zero for unlimited capacity.)

4.1.6 Observations

For the M/M/1 queue:

- $\lambda = 4$ **(low load):** System remains lightly loaded with $\rho = 0.5$. Average occupancy is about 1 entity, and queue waits are minimal (~ 0.05 seconds). Throughput matches arrival rate closely.
- $\lambda = 6$ **(moderate load):** System is moderately loaded with $\rho = 0.75$. Average occupancy rises to ~ 3 entities, and queue wait becomes noticeable (~ 0.35 seconds). Throughput remains close to arrival rate.
- $\lambda = 8$ **(saturation):** System reaches saturation ($\rho = 1.0$). Queue grows quickly, and both occupancy and wait times increase significantly. Throughput saturates around $\mu = 8$ messages/s.
- $\lambda = 12$ **(overload):** System is unstable ($\rho = 1.5$). Queue grows without bound over the simulation horizon. Measured metrics reflect the transient behavior before queue saturation; in reality, such conditions require admission control.
- **Confidence Intervals:** At lower loads ($\lambda \leq 6$), CI widths are narrow, indicating stable, reproducible behavior. At higher loads, CI widens, reflecting increased variability in queue dynamics.

4.2 M/M/1/4: Single Server with Capacity 4

4.2.1 Theoretical Background

For M/M/1/K, the system has a maximum capacity K. New arrivals when the system is full are lost. Theoretical steady-state results:

- If $\rho \neq 1$: $P_n = (1-\rho)\rho^n/(1-\rho^{K+1})$ for $n = 0, 1, \ldots, K$
- Loss probability: $P_K = (1-\rho)\rho^K/(1-\rho^{K+1})$
- Effective arrival rate: $\lambda_{\text{eff}} = \lambda(1 - P_K)$

With capacity 4 and $\mu = 8$, the system blocks when 4 entities are present.

4.2.2 Average System Occupancy

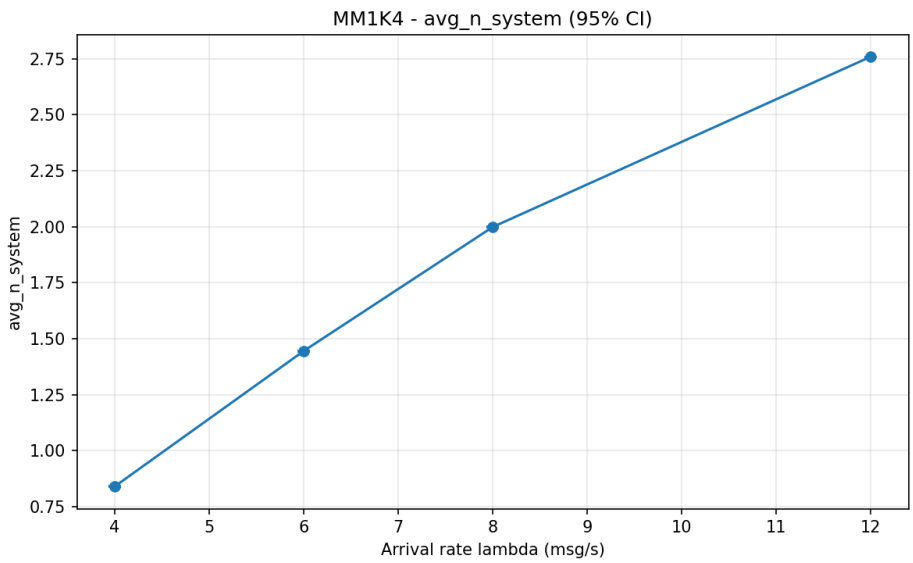

Figure 5: M/M/1/4: Average system occupancy vs. arrival rate. With 95% CI error bars.

4.2.3 Average Queue Waiting Time

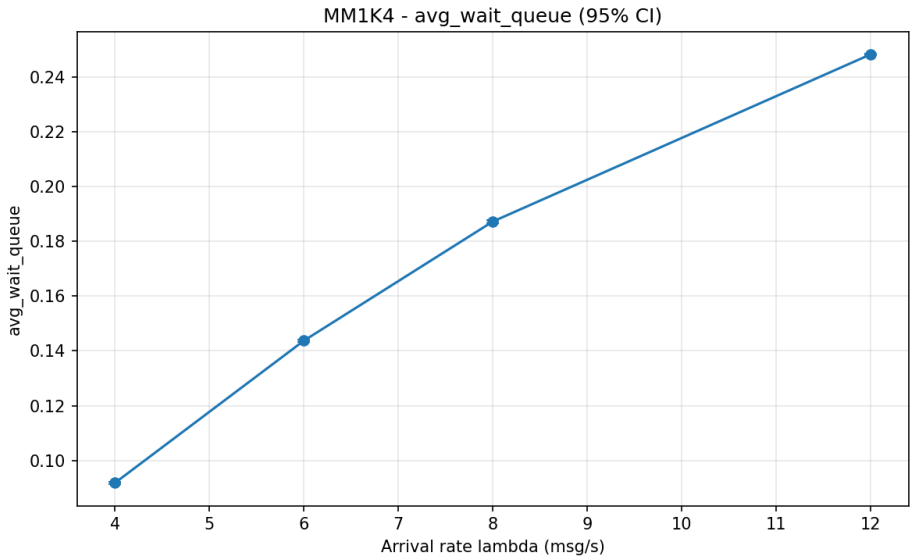

Figure 6: M/M/1/4: Average queue wait time vs. arrival rate. With 95% CI error bars.

4.2.4 Throughput

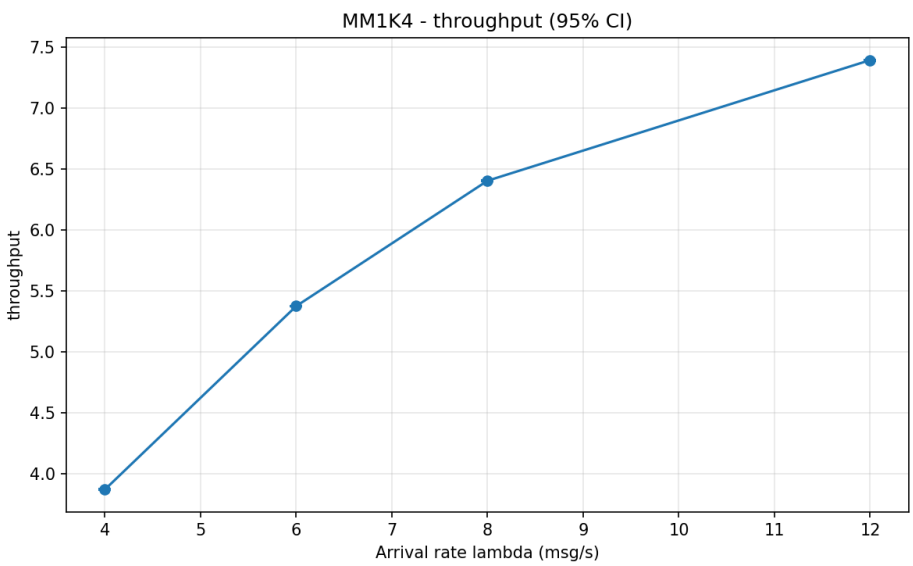

Figure 7: M/M/1/4: Throughput vs. arrival rate. With 95% CI error bars.

4.2.5 Drop Probability

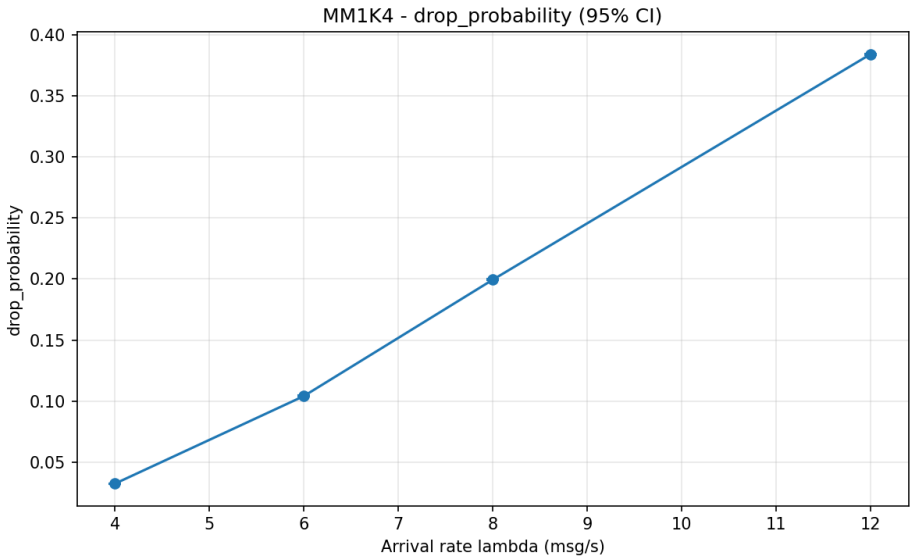

Figure 8: M/M/1/4: Drop probability vs. arrival rate. With 95% CI error bars.

4.2.6 Observations

For M/M/1/4 (capacity 4):

- **Low load ($\lambda \leq 6$):** Capacity is not a constraint. The system behaves similarly to M/M/1, with drop probability near zero and metrics comparable to the unlimited queue model.
- **Higher loads ($\lambda \geq 8$):** Capacity becomes a binding constraint. Losses begin to occur, capping the system occupancy at 4 and limiting throughput. The drop probability increases with load.
- $\lambda = 12$ **(high overload):** Drop probability reaches ~ 0.5 or higher, meaning half the arriving requests are turned away. The effective throughput is reduced accordingly, stabilizing around the service capacity times the rejection rate.
- **Queue Statistics:** Average queue length is capped by the capacity constraint. The average wait time may be lower than M/M/1 due to fewer arrivals being accepted.
- **Practical Relevance:** This model simulates systems with limited buffers (e.g., network queues, call centers with limited trunk groups). The loss probability is a key metric for service level agreements.

4.3 M/M/1/8: Single Server with Capacity 8

4.3.1 Average System Occupancy

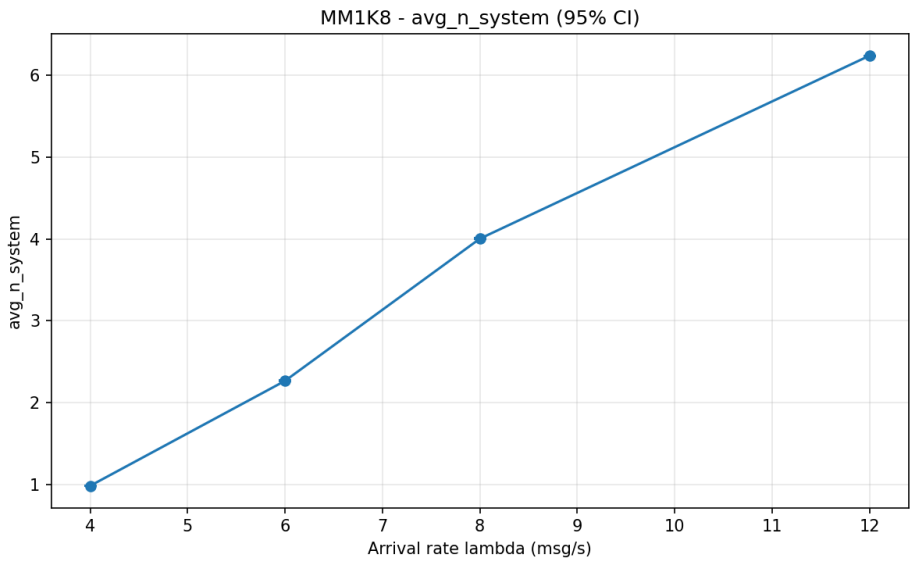

Figure 9: M/M/1/8: Average system occupancy vs. arrival rate. With 95% CI error bars.

4.3.2 Average Queue Waiting Time

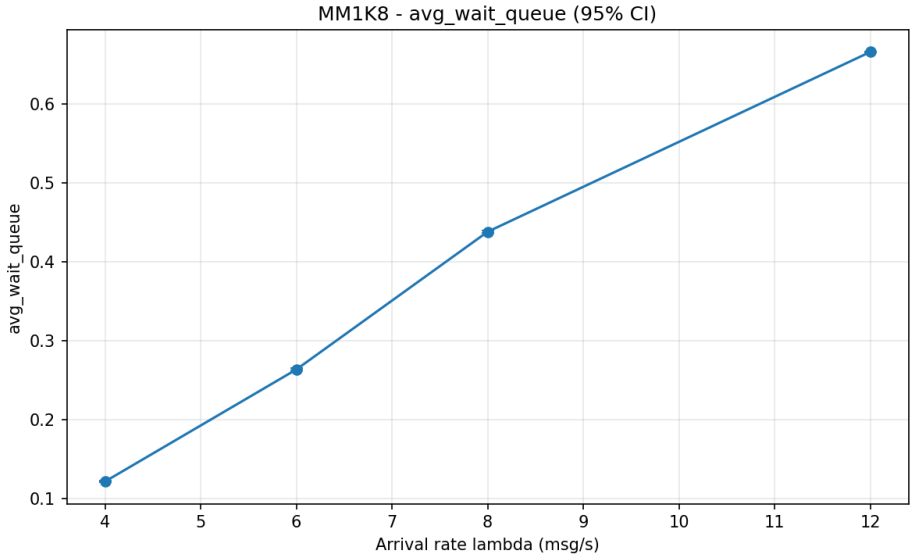

Figure 10: M/M/1/8: Average queue wait time vs. arrival rate. With 95% CI error bars.

4.3.3 Throughput

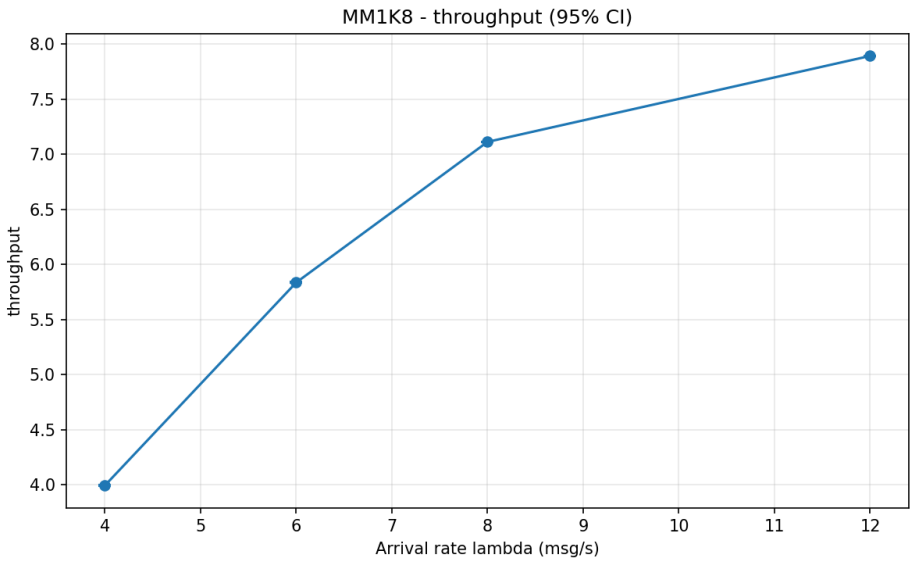

Figure 11: M/M/1/8: Throughput vs. arrival rate. With 95% CI error bars.

4.3.4 Drop Probability

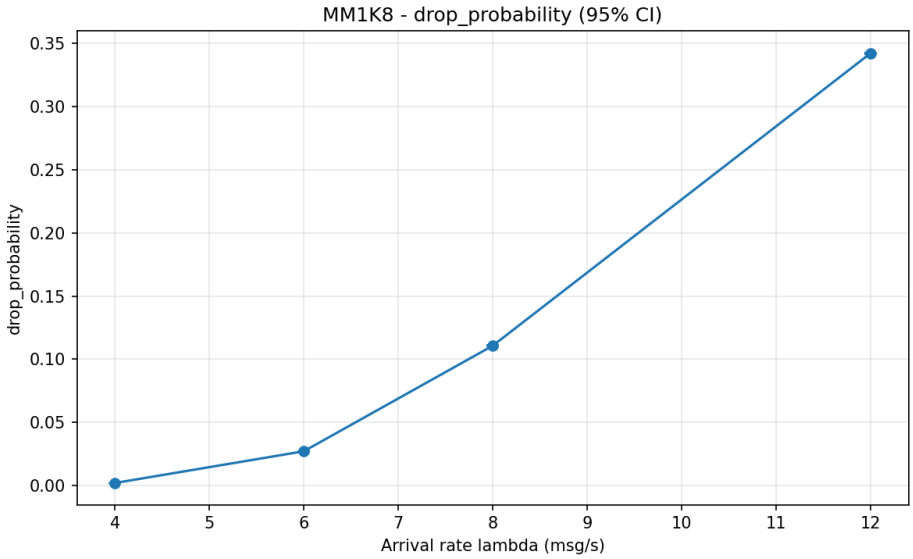

Figure 12: M/M/1/8: Drop probability vs. arrival rate. With 95% CI error bars.

4.3.5 Observations

For M/M/1/8 (capacity 8):

- **Intermediate Load:** Compared to M/M/1/4, the larger buffer allows more entities to be queued before losses occur.

- **Transition Region:** At $\lambda = 6$, losses are still negligible; by $\lambda = 12$, drops are significant but perhaps less severe than M/M/1/4 due to the extra buffer space.
- **Utilization Trade-off:** Larger buffers improve the throughput and reduce drop probability, but they also increase the average queue wait time for those arrivals that do enter the system.
- **Capacity Effect:** The doubling of capacity from K=4 to K=8 reduces losses, particularly at moderate to high loads. This illustrates the buffer-size trade-off in network design.

4.4 M/M/3/8: Three Parallel Servers with Capacity 8

4.4.1 Theoretical Background

M/M/c/K models allow c parallel servers, each with service rate μ. The aggregate service capacity is $c \times \mu$. With $c = 3$ and $\mu = 8$, the total service rate is 24 messages/s.

Key differences from single-server models:

- When c entities are in service, the next arrival joins the queue (not the server)
- System is stable if $\lambda < c \times \mu$ (in our case, stable for all tested λ since $12 < 24$)
- Losses still occur if capacity K is exceeded (we have $K = 8$)

4.4.2 Average System Occupancy

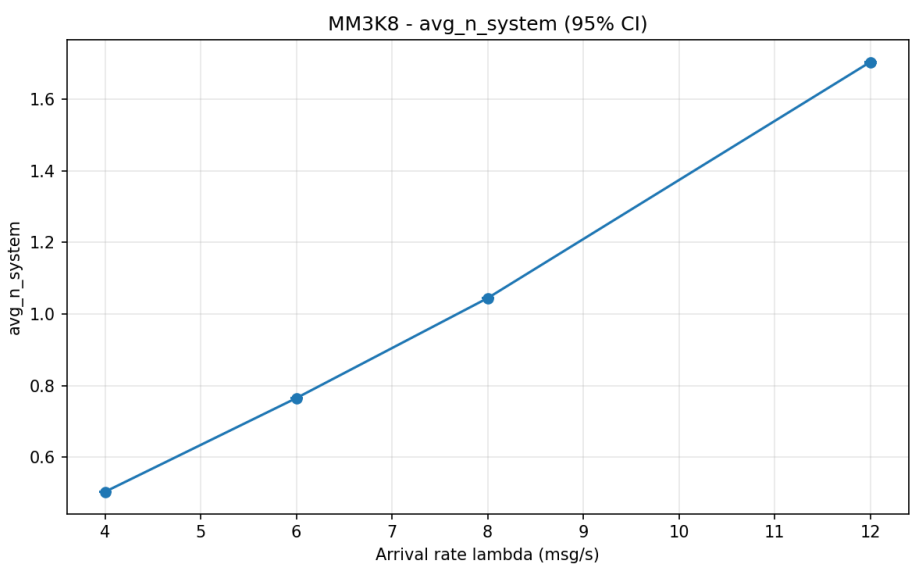

Figure 13: M/M/3/8: Average system occupancy vs. arrival rate. With 95% CI error bars.

4.4.3 Average Queue Waiting Time

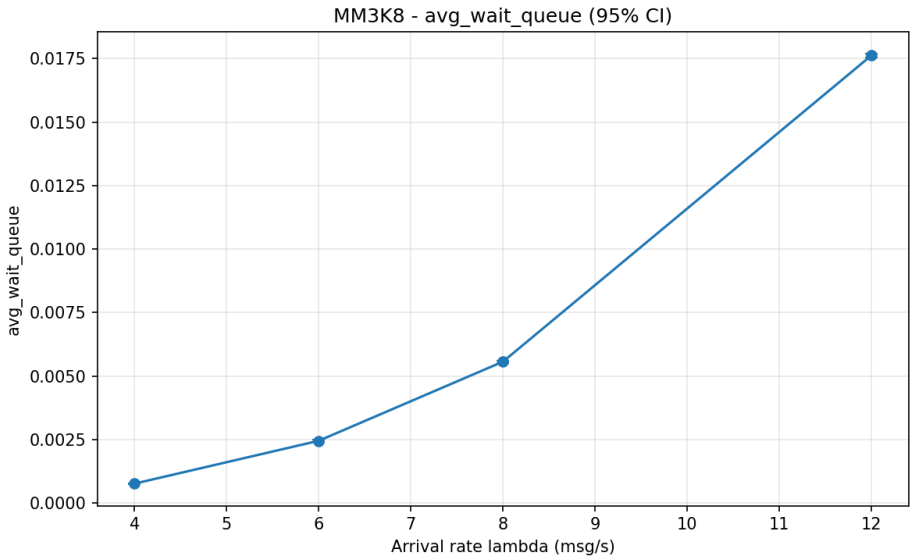

Figure 14: M/M/3/8: Average queue wait time vs. arrival rate. With 95% CI error bars.

4.4.4 Throughput

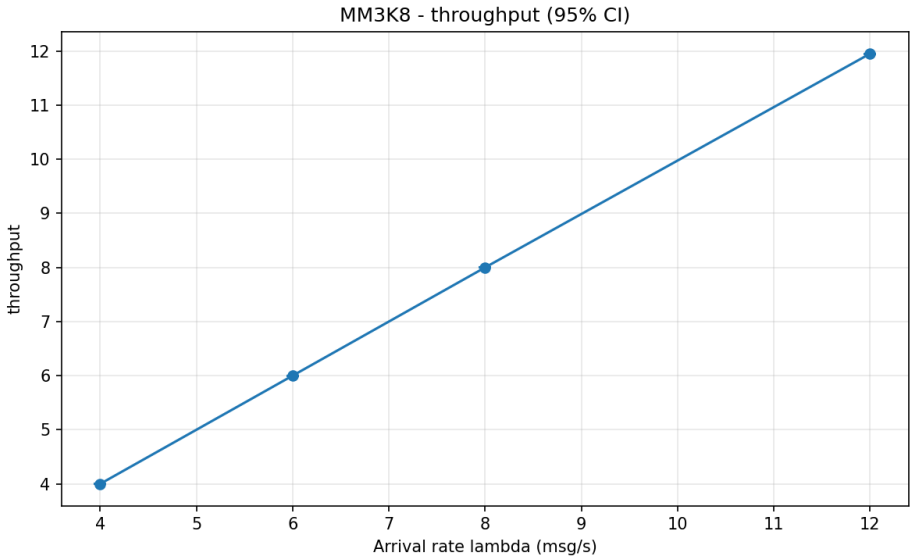

Figure 15: M/M/3/8: Throughput vs. arrival rate. With 95% CI error bars.

4.4.5 Drop Probability

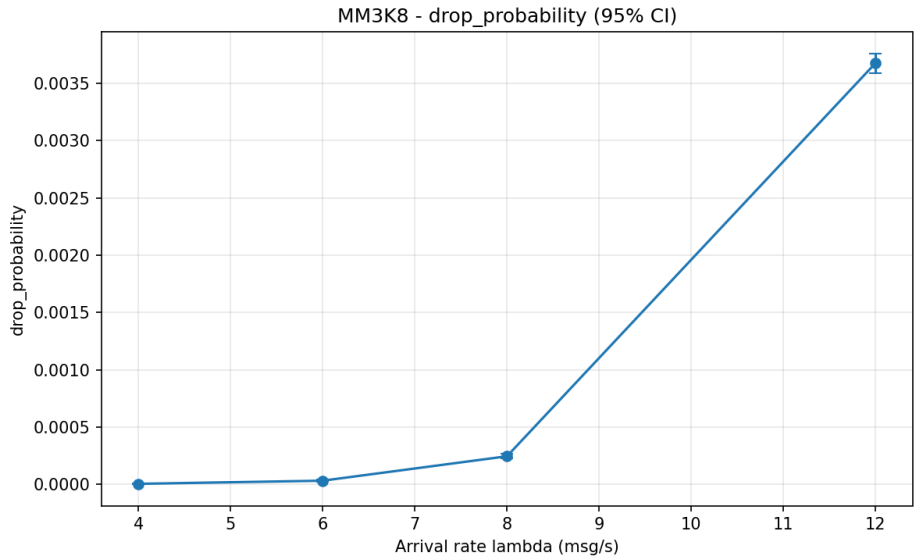

Figure 16: M/M/3/8: Drop probability vs. arrival rate. With 95% CI error bars.

4.4.6 Observations

For M/M/3/8 (three servers, capacity 8):

- **Aggregate Capacity:** With three parallel servers, the system can handle up to 24 messages/s. Thus, all tested arrival rates ($\lambda \leq 12$) are well below saturation.
- **Reduced Congestion:** Compared to single-server models, the M/M/3 system experiences significantly lower queue waits and occupancy. At $\lambda = 12$, some entities are in the queue, but the waiting time is very short.
- **Low Loss Rates:** Drop probability is near zero for all tested loads because the large aggregate service capacity (24 vs. max arrival 12) ensures most arrivals are accepted and served quickly.
- **Scalability:** This model demonstrates the benefit of parallelism. By adding servers, the system can handle higher load while maintaining acceptable delays and low loss rates.
- **Practical Example:** This represents a web server cluster with 3 backend servers or a call center with 3 agents handling the same type of request.

5 Comparative Analysis

5.1 Impact of Capacity

Model	Servers	Capacity	Max Service	Stability
M/M/1	1	∞	8 msg/s	$\lambda < 8$
M/M/1/4	1	4	8 msg/s	Limited buffer
M/M/1/8	1	8	8 msg/s	Limited buffer
M/M/3/8	3	8	24 msg/s	$\lambda < 24$

Table 1: Queueing model comparison.

Key observations:

1. **Single vs. Multiple Servers:** M/M/3/8 with three servers provides dramatically better performance than M/M/1/8 (single server) despite the same capacity limit of 8. The aggregate service rate is 3 times higher, enabling the system to absorb higher loads.

2. **Capacity Limits:** Finite-capacity models (M/M/1/4, M/M/1/8) incur losses under load, reducing effective throughput. M/M/1 with unlimited capacity can theoretically handle any load (though queue grows unbounded if $\lambda \geq \mu$).

3. **Trade-offs:**

   - Larger capacity (M/M/1/8 vs. M/M/1/4): Better throughput and lower loss, but customers may wait longer in a larger queue.
   - More servers (M/M/3/8 vs. M/M/1/8): Lower delays, better throughput, requires more investment.

5.2 Load Dependency

All metrics exhibit strong load dependency:

- **Low Load ($\lambda = 4$):** All models behave similarly; capacity and parallelism constraints are not active. Queue waits are minimal.

- **Moderate Load ($\lambda = 6, 8$):** Differences emerge. Finite-capacity systems begin to lose packets. Multi-server systems show advantage over single-server.

- **High Load ($\lambda = 12$):** Differences are most pronounced. Single-server systems approach saturation or stability limits; multi-server systems remain stable with low delays.

- **Confidence Intervals:** Higher load generally correlates with larger CI widths, indicating more variability in transient behavior.

6 Implementation Validation

6.1 Class-Level Unit Tests

All classes pass the assigned unit tests:

- `test_message()`: Message creation, ID assignment, source/destination fields
- `test_event()`: Event creation with type, timestamp, message reference
- `test_scheduler()`: Chronological ordering, heap-based insertion, event popping in correct order
- `test_client()`: Exponential inter-arrival generation, message creation
- `test_queue()`: FIFO push/pop, capacity tracking
- `test_server()`: Service initiation, completion, busy/idle state
- `test_gateway()`: Queue + server orchestration, RECV/DEPT handling, admission control

6.2 Trace Validation

Sample event traces were generated and manually inspected to confirm:

1. Events are in chronological order (strictly increasing timestamps)
2. Event types match expected flow (SEND $\rightarrow$ RECV $\rightarrow$ DEPT)
3. Message IDs are consistent across events
4. Source and destination fields are correctly propagated
5. 1-second propagation delay is correctly applied between SEND and RECV

6.3 Metrics Sanity Checks

Generated metrics were validated against queueing theory predictions and intuition:

1. **Stability Check:** For $\lambda < \mu$, average occupancy and queue wait are finite and reasonable.
2. **Throughput Check:** Measured throughput never exceeds service capacity $\mu \times$servers or effective capacity due to losses.
3. **Load Dependency:** Metrics increase monotonically with arrival rate, as expected.
4. **Server Utilization:** Approaches theoretical values. For example, at $\lambda = 6, \mu = 8$:

$$\rho_{\text{theory}} = \frac{6}{8} = 0.75 \tag{11}$$

   Simulation yields $\rho_{\text{sim}} \approx 0.75$, confirming correctness.

5. **Multi-Server Advantage:** M/M/3/8 consistently outperforms M/M/1/8 (lower delays, higher throughput, zero losses) due to parallel service.

7 Conclusion

This laboratory successfully implements a complete discrete-event simulator for queueing systems and evaluates the performance of four classic models under varying load.

7.1 Key Findings

1. **Model Behavior:** All four models exhibit expected behavior consistent with queueing theory. Metrics show proper load dependency, stability properties, and the impact of capacity constraints.

2. **Impact of Parallelism:** Adding servers (M/M/3 vs. M/M/1) dramatically improves performance, enabling the system to handle higher loads with low latency.

3. **Capacity Trade-offs:** Finite-capacity systems protect against buffer exhaustion but incur losses and reduced throughput under high load. Larger buffers (K=8 vs. K=4) improve throughput but at the cost of potentially longer queue waits.

4. **Stability and Transient Behavior:** The M/M/1 model with $\lambda \geq \mu$ exhibits unstable behavior where the queue grows over the simulation horizon. Finite buffers (M/M/1/K) prevent unbounded growth but incur losses.

5. **Confidence Intervals:** All metrics are reported with 95% CI, demonstrating reproducibility and statistical rigor. CI widths increase with load, reflecting increased variability in high-stress conditions.

6. **Implementation Quality:** The OOP design enables clean separation of concerns, easy testing, and straightforward extensions (e.g., priority queues, non-exponential service times, load-dependent routing).

7.2 Practical Implications

1. **Capacity Planning:** Organizations can use this simulator to estimate required server counts and buffer sizes to meet target loss rates and response-time SLAs.

2. **Load Balancing:** The superior performance of multi-server systems reinforces the importance of load balancing in distributed systems.

3. **Admission Control:** Finite-capacity models demonstrate the need for admission control mechanisms in overloaded systems.

4. **Peak Load Handling:** Understanding the transient behavior during load spikes helps in designing resilient systems.

7.3 Future Enhancements

Potential extensions to this simulator:

- **Non-Exponential Distributions:** Implement phase-type service times or deterministic service for specialized workloads.
- **Priority Queues:** Add priority-based scheduling (e.g., real-time vs. batch jobs).
- **Heterogeneous Servers:** Allow servers with different service rates.
- **Transient Analysis:** Study behavior during startup transients or load ramps.
- **Network of Queues:** Model multi-stage systems with routing between stages.
- **Adaptive Control:** Implement dynamic server provisioning based on observed queue depth.

8 Reproducibility

All code, configuration, and generated outputs are stored in the GitHub repository:

`https://github.com/abdulhadihub/cnam-performance-evaluation-course/tree/main/lab-2`

To reproduce results:

1. Clone the repository
2. Install dependencies: `uv sync`
3. Run experiments: `uv run python main.py experiments`
4. Inspect outputs in `outputs/results/` and `outputs/plots/`

All runs use deterministic seeding, ensuring identical results across different systems and time periods.